

Numerical Series & Patterns: 15 Solved Problems

Code + Line-by-Line Explanations

Each problem below is solved using only loops and conditional statements — no advanced libraries — with a full line-by-line walkthrough of the code.

1. Fibonacci Series up to N Terms

```
n = int(input("Enter number of terms: "))

a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Line-by-line: 1. `n = int(input(...))` — how many Fibonacci terms to print. 2. `a, b = 0, 1` — `a` holds the “current” term to print, `b` holds the “next” term. This is the starting pair of the sequence. 3. `for i in range(n):` — repeat exactly `n` times, once per term. 4. `print(a, end=" ")` — print the current term, staying on the same line. 5. `a, b = b, a + b` — this is the heart of the logic: `a` becomes what used to be `b` (the next term), and `b` becomes the sum of the *old* `a` and *old* `b` (the term after that). Because Python evaluates the right-hand side entirely before assigning, there’s no need for a separate temporary variable.

2. Sum of Natural Numbers Divisible by 3 or 5

```
n = int(input("Enter n: "))

total = 0
for num in range(1, n):
    if num % 3 == 0 or num % 5 == 0:
        total += num

print("Sum:", total)
```

Line-by-line: 1. `n = int(input(...))` — the upper limit (exclusive, per problem definition “below `n`”). 2. `total = 0` — accumulator, starts at zero. 3. `for num in range(1, n):` — check every natural number from 1 up to (but not including) `n`. 4. `if num % 3 == 0 or num % 5 == 0:` — `%` gives the remainder; a remainder of 0 means the number divides evenly. The `or` means we only need *one* of the two conditions to be true. 5. `total += num` — if the number qualifies, add it to the running total. 6. `print("Sum:", total)` — display the final accumulated sum.

3. Prime Numbers in a Range

```
a = int(input("Start of range: "))
b = int(input("End of range: "))

for num in range(a, b + 1):
    if num < 2:
        continue
    is_prime = True
    for divisor in range(2, int(num ** 0.5) + 1):
        if num % divisor == 0:
            is_prime = False
            break
```

```

    if is_prime:
        print(num, end=" ")

```

Line-by-line: 1-2. Read the start and end of the range to check. 3. `for num in range(a, b + 1):` — loop through every number in the range, inclusive of `b`. 4. `if num < 2: continue` — numbers less than 2 (0, 1, and negatives) are never prime by definition; `continue` skips straight to the next number. 5. `is_prime = True` — assume the current number is prime until proven otherwise. 6. `for divisor in range(2, int(num ** 0.5) + 1):` — only check potential divisors up to the square root of `num` — any factor larger than the square root would have a corresponding factor smaller than it, so checking further is redundant. 7. `if num % divisor == 0:` — if `divisor` evenly divides `num`, we've found a factor. 8. `is_prime = False` — the number is not prime. 9. `break` — no need to keep checking once we know it's not prime. 10. `if is_prime: print(num, end=" ")` — after the inner loop finishes (whether by exhausting all divisors or breaking early), print the number if it survived as prime.

4. Armstrong Numbers in a Range (1-1000)

```

for num in range(1, 1001):
    digits = str(num)
    power = len(digits)
    total = 0
    for d in digits:
        total += int(d) ** power
    if total == num:
        print(num, end=" ")

```

Line-by-line: 1. `for num in range(1, 1001):` — check every number from 1 to 1000. 2. `digits = str(num)` — convert the number to a string so we can iterate over individual digit characters. 3. `power = len(digits)` — the number of digits determines the exponent used (e.g., a 3-digit number uses power 3). 4. `total = 0` — accumulator for the sum of `digit ** power`. 5. `for d in digits:` — loop through each digit character. 6. `total += int(d) ** power` — convert the character back to an integer, raise it to `power`, and add it to the running total. 7. `if total == num:` — an Armstrong number is one where this sum equals the original number. 8. `print(num, end=" ")` — display it if the condition holds.

5. Perfect Numbers up to N

```

n = int(input("Enter n: "))

for num in range(2, n + 1):
    divisor_sum = 0
    for i in range(1, num):
        if num % i == 0:
            divisor_sum += i
    if divisor_sum == num:
        print(num, end=" ")

```

Line-by-line: 1. Read the upper limit `n`. 2. `for num in range(2, n + 1):` — check every candidate number starting from 2 (1 has no proper divisors, so it can be skipped). 3. `divisor_sum = 0` — accumulator for this candidate's proper divisors. 4. `for i in range(1, num):` — check every number from 1 up to (but not including) `num` itself — these are the *proper* divisors (excluding the number itself). 5. `if num % i == 0:` — if `i` divides `num` evenly, it's a proper divisor. 6. `divisor_sum += i` — add it to the running sum. 7. `if divisor_sum == num:` — a perfect number's proper divisors sum to exactly itself. 8. `print(num, end=" ")` — display it if the condition holds.

(Note: this is the simple, easy-to-follow version; checking divisors only up to `sqrt(num)` and adding both `i` and `num//i` at once would be a faster, more advanced refinement.)

6. Floyd's Triangle

```
n = int(input("Enter number of rows: "))

current_number = 1
for row in range(1, n + 1):
    for col in range(row):
        print(current_number, end=" ")
        current_number += 1
    print()
```

Line-by-line: 1. Read the number of rows to print. 2. `current_number = 1` — a counter that persists **across** all rows (unlike a typical nested-loop pattern where the inner value resets each row). 3. `for row in range(1, n + 1):` — outer loop controls how many rows are printed. 4. `for col in range(row):` — inner loop runs `row` times — row 1 prints once, row 2 prints twice, and so on. 5. `print(current_number, end=" ")` — print the current counter value, staying on the same line. 6. `current_number += 1` — increment the shared counter after every single number printed (not just once per row) — this is what makes the numbers keep counting upward continuously across the whole triangle. 7. `print()` — after finishing a row’s inner loop, move to a new line before starting the next row.

7. Pascal’s Triangle

```
n = int(input("Enter number of rows: "))

for row in range(n):
    value = 1
    for col in range(row + 1):
        print(value, end=" ")
        value = value * (row - col) // (col + 1)
    print()
```

Line-by-line: 1. Read the number of rows. 2. `for row in range(n):` — outer loop, one iteration per row (0-indexed). 3. `value = 1` — every row starts with 1 (the leftmost entry of any row in Pascal’s Triangle is always 1). 4. `for col in range(row + 1):` — a row `row` (0-indexed) has exactly `row + 1` entries. 5. `print(value, end=" ")` — print the current entry. 6. `value = value * (row - col) // (col + 1)` — this formula computes the *next* entry in the row directly from the current one, using the mathematical relationship between adjacent binomial coefficients ($C(row, col+1) = C(row, col) * (row - col) / (col + 1)$) — avoiding the need to recompute factorials from scratch for every entry. 7. `print()` — move to a new line after finishing the row.

8. Collatz Sequence Length

```
n = int(input("Enter starting number: "))
steps = 0
current = n

while current != 1:
    if current % 2 == 0:
        current = current // 2
    else:
        current = current * 3 + 1
    steps += 1

print("Steps to reach 1:", steps)
```

Line-by-line: 1. Read the starting number. 2. `steps = 0` — counts how many transformations have been applied. 3. `current = n` — a working copy of the number, since we don’t want to overwrite the original input. 4. `while current != 1:` — keep applying the rule until we reach 1 (the sequence is conjectured to always eventually reach 1, though this is technically an unproven mathematical conjecture — the “Collatz Conjecture”). 5. `if current % 2 == 0:` — check whether the current value is even. 6. `current = current // 2` — if even, halve it (integer division since the result is always exact for even numbers). 7. `else: current = current * 3 + 1` — if odd, apply the “ $3n+1$ ” rule. 8. `steps += 1` — count this transformation, regardless of which branch was taken. 9. Once the loop ends (value reached 1), print the total step count.

9. Sum of a Series: 1 - 2 + 3 - 4 + 5 ... up to N Terms

```
n = int(input("Enter number of terms: "))

total = 0
for i in range(1, n + 1):
    if i % 2 == 0:
        total -= i
    else:
        total += i

print("Sum:", total)
```

Line-by-line: 1. Read how many terms to include. 2. `total = 0` — accumulator for the alternating sum. 3. `for i in range(1, n + 1):` — loop through terms 1 to n. 4. `if i % 2 == 0:` — even-positioned terms (2nd, 4th, 6th...) get subtracted, per the series pattern. 5. `total -= i` — subtract this term. 6. `else: total += i` — odd-positioned terms (1st, 3rd, 5th...) get added. 7. `print("Sum:", total)` — display the final alternating sum.

(A more compact alternative replaces the if-else with `total += i * (-1) ** (i + 1)`, using the mathematical fact that (-1) raised to alternating powers flips sign every term — worth showing once the loop-based logic is understood.)

10. Happy Number Checker

```
n = int(input("Enter a number: "))

seen = set()
current = n

while current != 1 and current not in seen:
    seen.add(current)
    total = 0
    for digit in str(current):
        total += int(digit) ** 2
    current = total

print("Happy number!" if current == 1 else "Not a happy number.")
```

Line-by-line: 1. Read the number to check. 2. `seen = set()` — tracks every value we've already visited, to detect when we're stuck in a repeating cycle (a sign the number is *not* happy). 3. `current = n` — working copy of the number. 4. `while current != 1 and current not in seen:` — keep transforming as long as we haven't reached 1 (success) AND haven't seen this value before (which would mean we're looping forever without success). 5. `seen.add(current)` — record this value before transforming it, so future repeats can be detected. 6. `total = 0` — accumulator for the sum of squared digits. 7. `for digit in str(current):` — loop through each digit of the current value. 8. `total += int(digit) ** 2` — square each digit and add it to the total. 9. `current = total` — the new "current" value becomes this sum, ready for the next iteration. 10. After the loop ends (either reaching 1, or detecting a repeat), report the result based on which condition caused the exit.

11. GCD (HCF) of Two Numbers Using a Loop

```
a = int(input("First number: "))
b = int(input("Second number: "))

smaller = min(a, b)
gcd = 1

for i in range(1, smaller + 1):
    if a % i == 0 and b % i == 0:
        gcd = i
```

```
print("GCD:", gcd)
```

Line-by-line: 1-2. Read the two numbers. 3. `smaller = min(a, b)` — the GCD can never be larger than the smaller of the two numbers, so we only need to check divisors up to this value. 4. `gcd = 1` — a safe default (1 always divides evenly into everything, so this guarantees a valid answer even in the worst case). 5. `for i in range(1, smaller + 1):` — check every candidate divisor from 1 up to `smaller`. 6. `if a % i == 0 and b % i == 0:` — check if `i` divides *both* numbers evenly. 7. `gcd = i` — if so, update our answer — since we’re scanning in increasing order, the *last* one that matches will be the *largest* common divisor by the time the loop finishes. 8. Print the final result.

(Note: this is the simple, easy-to-follow $O(n)$ version. The Euclidean algorithm, using `while b: a, b = b, a % b`, solves this in far fewer steps — a good “next level” refinement to show once this version is understood.)

12. Look-and-Say Sequence

```
n = int(input("Which term to generate? "))

term = "1"
for _ in range(n - 1):
    next_term = ""
    i = 0
    while i < len(term):
        count = 1
        while i + 1 < len(term) and term[i] == term[i + 1]:
            count += 1
            i += 1
        next_term += str(count) + term[i]
        i += 1
    term = next_term

print("Term", n, ":", term)
```

Line-by-line: 1. Read which term number to generate (term 1 is always “1”). 2. `term = "1"` — start with the first term of the sequence. 3. `for _ in range(n - 1):` — apply the “describe the previous term” transformation `n - 1` times to go from term 1 to term `n`. 4. `next_term = ""` — builds the newly described term. 5. `i = 0` — a manually controlled index into the current `term`, since we need to jump forward by variable amounts (unlike a simple `for char in term`). 6. `while i < len(term):` — process the current term from start to finish. 7. `count = 1` — every run of repeated digits starts with a count of at least 1 (the digit itself). 8. `while i + 1 < len(term) and term[i] == term[i + 1]:` — keep checking: is there a *next* character, and does it match the current one? 9. `count += 1` — extend the run count. 10. `i += 1` — move forward to check the character after that. 11. `next_term += str(count) + term[i]` — once the run ends, describe it as “count” followed by “the digit” (e.g., three consecutive 1s become “31”), and append this description to the result. 12. `i += 1` — move past this entire run to start processing the next distinct run of digits. 13. `term = next_term` — the newly built description becomes the current term for the next round of the outer loop. 14. After all `n - 1` transformations, print the final term.

13. Number Pyramid with Reverse Counting

```
n = int(input("Enter number of rows: "))

for row in range(1, n + 1):
    for num in range(1, row + 1):
        print(num, end=" ")
    for num in range(row - 1, 0, -1):
        print(num, end=" ")
    print()
```

Line-by-line: 1. Read the number of rows. 2. `for row in range(1, n + 1):` — outer loop, one iteration per row. 3. `for num in range(1, row + 1):` — first inner loop counts *up* from 1 to the current row number. 4. `print(num, end=" ")` — prints each ascending number, staying on the same line. 5. `for num in range(row - 1, 0, -1):` — second

inner loop counts back *down*, starting just below the row number (to avoid repeating the peak value) down to 1. 6. `print(num, end=" ")` — prints each descending number. 7. `print()` — move to a new line once both the ascending and descending halves of the row are complete.

14. Kaprekar's Routine (6174 Constant)

```
n = int(input("Enter a 4-digit number: "))

steps = 0
current = n

while current != 6174 and current != 0:
    digits = list(str(current).zfill(4))
    ascending = int("".join(sorted(digits)))
    descending = int("".join(sorted(digits, reverse=True)))
    current = descending - ascending
    steps += 1
    if steps > 20: # safety cap in case of a repeated-digit edge case (e.g., 1111)
        break

if current == 6174:
    print("Reached 6174 in", steps, "steps")
else:
    print("Does not converge (likely all digits are the same)")
```

Line-by-line: 1. Read the starting 4-digit number. 2. `steps = 0` — counts how many iterations of the routine have been applied. 3. `current = n` — working copy. 4. `while current != 6174 and current != 0:` — keep iterating until we either reach the famous constant 6174, or hit 0 (a sign of a degenerate all-same-digit input, like 1111, where ascending and descending arrangements are identical). 5. `digits = list(str(current).zfill(4))` — convert to a 4-character string (padding with leading zeros via `zfill(4)` in case leading digits become 0 partway through, e.g., 0999), then to a list of individual digit characters. 6. `ascending = int("".join(sorted(digits)))` — sort the digits in increasing order, join back into a string, and convert to an integer. 7. `descending = int("".join(sorted(digits, reverse=True)))` — same idea, but sorted in decreasing order. 8. `current = descending - ascending` — apply Kaprekar's subtraction rule. 9. `steps += 1` — count this iteration. 10. `if steps > 20: break` — a safety cap; if the input has all identical digits (like 1111), `descending - ascending` will always be 0, and the loop would otherwise need a separate exit check — this line guards against unexpected infinite loops from edge cases. 11–14. Report whether we successfully reached 6174 (with the step count) or hit the degenerate case.

15. Sum of Digits Raised to a Given Power (Generalized Armstrong Check)

```
num = int(input("Enter a number: "))
power = int(input("Enter the power to raise each digit to: "))

total = 0
for digit in str(num):
    total += int(digit) ** power

if total == num:
    print(f"{num} matches the sum of its digits raised to power {power}")
else:
    print(f"{num} does NOT match (sum was {total})")
```

Line-by-line: 1. Read the number to check. 2. Read the power to raise each digit to (this generalizes the classic Armstrong check, which always uses “power = number of digits”). 3. `total = 0` — accumulator. 4. `for digit in str(num):` — loop through each digit character of the number. 5. `total += int(digit) ** power` — convert to an integer, raise to the given power, and accumulate. 6. `if total == num:` — check whether the accumulated sum matches the original number. 7–8. Report the result, including the computed sum for transparency even when it doesn't match.